B5 Data validation

Schema checks with pandera and input models with pydantic.

B5-L1 DataFrame schemas with pandera

A schema at the point of entry turns a silent bad input into one loud, detailed failure.

```python
import pandera.pandas as pa

RESPONSES = pa.DataFrameSchema(
    {
        "respondent_id": pa.Column(str, unique=True),
        "age": pa.Column(int, pa.Check.in_range(16, 99)),
        "weight": pa.Column(float, pa.Check.gt(0)),
    },
    strict=True,
    coerce=True,
)
clean = RESPONSES.validate(raw, lazy=True)
```

pandera Column defaults	nullable=False. Set nullable=True where nulls are expected.
strict=True and coerce=True	strict rejects unknown columns. coerce converts dtypes before checks.
validate(df, lazy=True)	Runs every check and raises SchemaErrors with failure_cases.
SchemaError vs SchemaErrors	Separate classes. SchemaErrors is not a subclass. Catch both, or better, let them raise.

B5-L2 pydantic models for config and input records

Validate external settings and records once, at the boundary, with every rule in the model.

```python
from pydantic import BaseModel, ConfigDict, Field


class WeightingConfig(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)

    weight_col: str
    min_base: int = Field(default=50, gt=0)
    cap: float = Field(default=5.0, gt=1.0)


cfg = WeightingConfig.model_validate(yaml.safe_load(text))
```

pydantic extra keys, default behaviour	Ignored silently. Use ConfigDict(extra="forbid") for config.
pydantic lax mode	Converts "30" and 30.0 to 30, True to 1. Use strict=True to refuse conversions.
model_copy(update=...)	Does not validate. Rebuild with model_validate({**m.model_dump(), ...}).
pydantic or pandera	pydantic for records and settings from outside. pandera for whole DataFrames.

B5-L3 Validate at the boundaries, with severity

Check at boundaries. Collect structured results. Log them all, then stop on CRITICAL.

```python
Severity = Literal["CRITICAL", "WARN"]
```

```
2
3
4 @dataclass(frozen=True)
5 class QaResult:
6     check: str
7     severity: Severity
8     passed: bool
9     observed: float | None
```

Where to validate	At boundaries: after reading input, before writing output.
CRITICAL vs WARN	CRITICAL stops the run (raise). WARN records and continues.
Order of QA operations	Run every check, log all results, then raise if any CRITICAL failed.
Two coding rules for checks	Guard every division. Test for None with is None, never truthiness.